

Chapter 8

Context-free Grammars

So far we have studied the class of languages known as regular languages. They can be expressed using a finite automata or regular expressions (both are equivalent). Also we have shown the existence of non-regular languages.

8.1 Context-Free Grammar

- Aim is to recognize more languages.
- Context-free grammars are a model to recognize languages.
- Recognizes all regular languages (needs proof).
- Recognizes some non-regular languages.
- Has applications in programming languages and compiler design.

A context-free grammar consists of the following components:

- *Terminals*: Symbols of the alphabet.
- *Variables*: A second type of symbols that can be replaced. Usually represented with capital letters. Generates a set of strings.
- *Substitution rules* (or *productions* or simply *rules*): Rules that dictate how a variable gets replaced. The left hand side has the variable that is to be replaced and the right hand side has replacement “content”.
- *Start variable*: A special variable from which the process starts.

8.1.1 Process to generate strings from a given grammar

1. Write the start variable.
2. Pick a variable in the current string and replace it with one of its substitution rules.
3. Continue step 2 until no more variable remains in the current string (has only terminals).

Let us look at an example. Consider the following context-free grammar (in short CFG).

$$\begin{aligned} S &\longrightarrow 0S1 \\ S &\longrightarrow \epsilon \end{aligned}$$

The grammar has two terminals, 0 and 1. It has only one variable S , hence it is also the start variable. There are two productions rules for S as seen above.

Now let us see how a string gets generated.

$$S \longrightarrow 0S1 \longrightarrow 00S11 \longrightarrow 000S111 \longrightarrow 000111$$

In the first three steps we replace S by its first substitution rule, whereas in the last step we replace S by its second substitution rule. Since the last “string” has only terminals, the process stops. This generates the string 0^31^3 .

What other strings can this grammar generate?

All strings of the form 0^n1^n for $n \geq 0$.

Can it generate any strings not of the form 0^n1^n ?

No.

Therefore the language of the grammar is

$$L = \{0^n1^n \mid n \geq 0\}.$$

We know that L is a non-regular language. Hence CFGs accept languages that are not regular.

Note 6. A *derivation* of a string is a sequence of substitutions, starting from the start variable that produces the string. The *language of a grammar* is the set of all strings that can be derived from the start symbol.

Another example:

$$\begin{aligned} S &\longrightarrow ASB \mid \epsilon \\ A &\longrightarrow a \\ B &\longrightarrow bb \end{aligned}$$

The language generated by the above grammar is

$$L = \{a^n b^{2n} \mid n \geq 0\}$$

which is also not regular.

Exercise 14. What happens if we add the production rule

$$B \longrightarrow \epsilon$$

to the above grammar?

8.1.2 Formal definition

Definition 8.1.1. A *context-free grammar* is the 4-tuple (V, Σ, P, S) , where

- V is a finite set called variables,
- Σ is a finite set disjoint from V called terminals,
- $P \subseteq V \times \{V \cup \Sigma\}^*$ is a finite set of production rules, and
- $S \in V$ is the start variable.

Note that in the above definition, P is a relation since the same variable can have several production rules.

We next define some more notations and terminology related to CFGs. Let $G = (V, \Sigma, P, S)$ be a CFG.

- If $A \in V$, $u, v, w \in \{V \cup \Sigma\}^*$ and $A \rightarrow w$ is a rule, then we say that uAv *yields* uwv in one step. This is denoted as $uAv \Rightarrow uwv$.
- For $u, v \in (V \cup \Sigma)^*$, we say that u yields v , denoted as $u \xRightarrow{*} v$, if
 - $u = v$, or
 - there exists strings $u_0, u_1, \dots, u_k$ (for $k \geq 1$) in $\{V \cup \Sigma\}^*$, such that $u_0 = u$, $u_k = v$ and $u_{i-1} \Rightarrow u_i$ for all $1 \leq i \leq k$.

- The language of G ,

$$L(G) = \{w \in \Sigma^* \mid S \xRightarrow{*} w\}.$$

- A language $L \subseteq \Sigma^*$ is called a *context-free language* (CFL in short) if there is a CFG G , such that $L = L(G)$.

8.1.3 Examples of CFGs

1. $L_1 = \{a^n b^m \mid n \geq 0, n \leq m \leq 2n\}$

CFG for L_1 :

$$\begin{aligned} S &\rightarrow ASB \mid \epsilon \\ A &\rightarrow a \\ B &\rightarrow bb \mid b \end{aligned}$$

Intuition: Since the number of b 's can be any number between n and $2n$, we have two variables B_1 and B_2 for every occurrence of A . B_1 produces only b hence there are n b 's corresponding to B_1 . B_2 produces b and ϵ , hence there are k b 's corresponding to B_2 , where $0 \leq k \leq n$, is the number of times the rule $B_2 \rightarrow b$ is used.

Exercise 15. Construct CFGs for the following languages:

- (a) $L = \{a^n b^m \mid n \geq 0, 3n \leq m \leq 5n\}$
- (b) $L = \{a^n b^m \mid n \geq 0, n < m \leq 2n\}$

$$(c) L = \{a^n b^m c^l \mid m = n + l, l, m, n \geq 0\}$$

$$2. L_2 = \{w \in \{0, 1\}^* \mid \#_0(w) = \#_1(w)\}$$

CFG for L_2 :

$$S \longrightarrow 1S0S \mid 0S1S \mid \epsilon$$

Intuition: Divide all non-empty strings in L_2 into two cases: strings that begin with a 0 and strings that begin with a 1. Let w be a string in first case. Then w is of the form, $w = 0w_11w_2$, where w_1 and w_2 are strings with equal number of 0's and 1's (can be either of the two cases). Here the 1 between w_1 and w_2 is the 'matching' 1 for the starting 0. The second case is symmetric.

3. A string of parentheses is balanced if each left parenthesis has a matching right parenthesis and the matching pairs are well nested. Consider the following language consisting of balanced parentheses:

$$L_3 = \{w \in \{[,]\}^* \mid w \text{ is balanced}\}$$

CFG for L_3 :

$$S \longrightarrow [S] \mid SS \mid \epsilon$$

Intuition: Divide all non-empty strings in L_2 into two cases: strings that begin with a 0 and strings that begin with a 1. Let w be a string in first case. Then w is of the form, $w = 0w_11w_2$, where w_1 and w_2 are strings with equal number of 0's and 1's (can be either of the two cases). Here the 1 between w_1 and w_2 is the 'matching' 1 for the starting 0. The second case is symmetric.

Exercise 16. Construct a CFG for the following language:

$$\text{PALINDROMES} = \{w \in \{a, b\}^* \mid w = \text{rev}(w)\}$$

8.1.4 Regular languages are context-free

Let L be a regular language accepted via a DFA $D = (Q, \Sigma, \delta, q_0, F)$. We will construct a CFG $G = (V, \Sigma, P, R_0)$ for L .

- $V = \{R_i \mid q_i \in Q, \forall i\}$. In other words, there is a variable in the CFG corresponding to every state in the DFA.
- The set of terminals is the same as the alphabet of D .
- R_0 is the start variable, that is, the variable corresponding to the start state.
- If $\delta(q_i, a) = q_j$, then the substitution rule $R_i \longrightarrow aR_j$ is in P .
- Also if $q_i \in F$, then $R_i \longrightarrow \epsilon$ is a substitution rule in P .

Lemma 12. $L(D) = L(G)$

Proof. First we will show that $L(D) \subseteq L(G)$.

Claim 13. For $w \in \Sigma^*$, if $\delta(q_0, w) = q_i$ in D then $R_0 \xRightarrow{*} wR_i$ in G .

Proof. We will prove by induction on the length of w . If $w = \epsilon$, then the claim follows from the definition of $\xRightarrow{*}$. Let $w = xa$ where $a \in \Sigma$. Then, $\delta(q_0, w) = \delta(q_0, xa) = \delta(\delta(q_0, x), a) = q_i$ (say). Let $\delta(q_0, x) = q_j$. By induction hypothesis we have $R_0 \xRightarrow{*} xR_j$. Also since $\delta(q_j, a) = q_i$, we have the substitution rule $R_j \rightarrow aR_i$ in G . Therefore, $R_0 \xRightarrow{*} xaR_i$. $\square$

Now if $w \in L(D)$, then $q_i \in F$, which implies that $R_i \rightarrow \epsilon$ is a substitution rule, and therefore $R_0 \xRightarrow{*} w$.

We will now show that $L(G) \subseteq L(D)$.

Claim 14. For $w \in \Sigma^*$, if $R_0 \xRightarrow{*} wR_i$ in G , then $\delta(q_0, w) = q_i$ in D .

Proof. We will prove by induction on the derivation length of w . If length is 0 then the claim follows trivially. Let $w = xa$ where $a \in \Sigma$. If $R_0 \xRightarrow{*} xaR_i$ then $R_0 \xRightarrow{*} xR_j$ and $R_j \rightarrow aR_i$ is a substitution rule. This is because of the type of substitution rules that G has (a terminal followed by a variable on the right hand side). BY induction hypothesis we have $\delta(q_0, x) = q_j$ and since $R_j \rightarrow aR_i$ is a substitution rule, we have $\delta(q_j, a) = q_i$. Therefore, $\delta(q_0, xa) = q_i$. $\square$

Now if $w \in L(G)$, then $R_i \rightarrow \epsilon$ is a substitution rule, which implies that $q_i \in F$, and therefore $\delta(q_0, w) \in F$.

Hence the lemma follows. $\square$

8.2 Ambiguity

8.2.1 Parse Trees

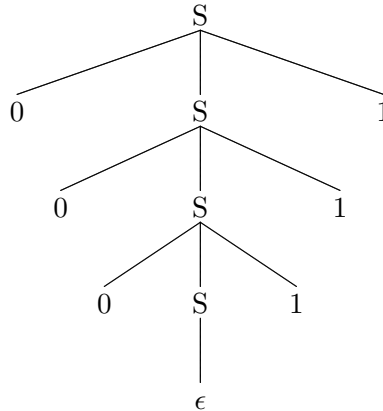
A *parse tree* for a string w with respect to a CFG G is a rooted, ordered tree that represents the derivation of w with respect to the grammar. It represents the “syntactic” structure of the string.

Examples of Parse Trees

1. Consider the CFG, say G_1 ,

$$S \rightarrow 0S1 \mid \epsilon$$

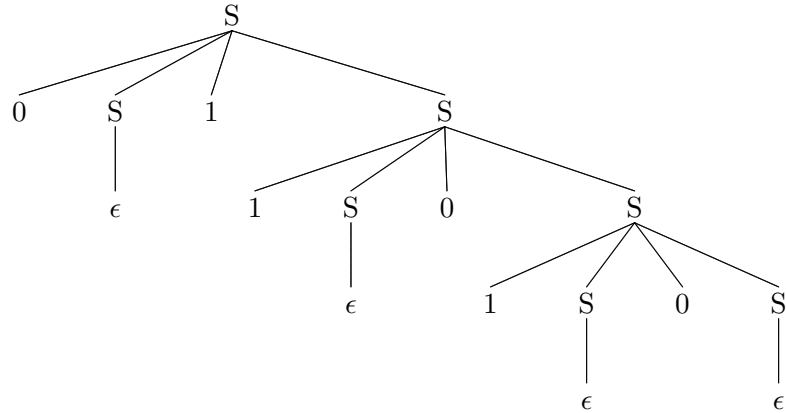
The parse tree of the string 000111 with respect to the above grammar is



2. Now consider the CFG, say G_2 ,

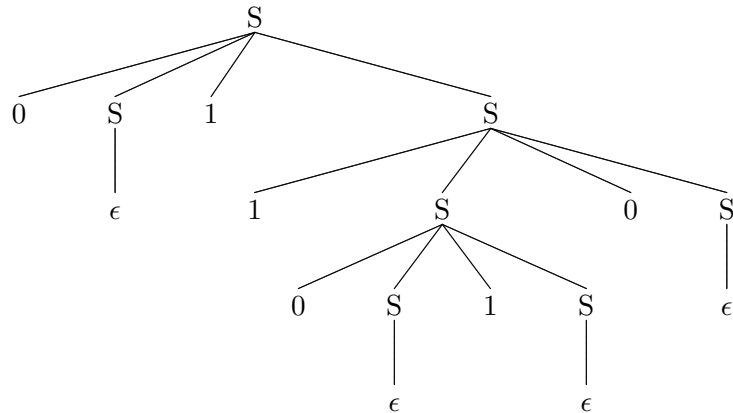
$$S \rightarrow 1S0S \mid 0S1S \mid \epsilon$$

The parse tree of the string 011010 with respect to the above grammar is



Note that in a parse tree concatenating the symbols from left to right gives the original string.

Another parse tree for the *same* string with respect to the *same* grammar.



Observe that in the first example there is a unique parse tree for every string in the language. But in the second example we have a string in the language that has two parse trees.

Remark. Given a CFG G , there can exist multiple parse trees for a string $w \in L(G)$.

Here are some properties of a parse tree that will be useful for us later on.

1. Every internal node of the tree is labelled by a variable.
2. Every leaf node is labelled either by a terminal or by ϵ . Moreover, if a leaf is labelled ϵ then it is the only child of its parent.
3. If an internal node is labelled A and its children are labelled $X_1, X_2, \dots, X_k$ in order from left to right, then $A \rightarrow X_1X_2 \dots X_k$ is a rule in the CFG.

8.2.2 Derivation and Ambiguity

The *derivation* of a string w with respect to a CFG is a sequence of substitutions that yields w starting from the start variable.

Examples

1. Derivation of 000111 with respect to G_1 .

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111$$

Observe that the string 000111 has only one derivation with respect to G_1 , because the string generated in each intermediate step has exactly one variable. In fact every string in $L(G_1)$ has exactly one derivation. However this need not be the case with every grammar as we shall see.

2. Consider the string 0101 in the language of the CFG G_2 . Below we show 3 derivations of the string with respect to G_2 .

$$\text{Derivation 1: } \underline{S} \Rightarrow 0\underline{S}1S \Rightarrow 01\underline{S} \Rightarrow 010\underline{S}1S \Rightarrow 0101\underline{S} \Rightarrow 0101$$

$$\text{Derivation 2: } \underline{S} \Rightarrow 0S1\underline{S} \Rightarrow 0S10\underline{S}1S \Rightarrow 0S101\underline{S} \Rightarrow 0\underline{S}101 \Rightarrow 0101$$

$$\text{Derivation 3: } \underline{S} \Rightarrow 0\underline{S}1S \Rightarrow 01\underline{S}0S1S \Rightarrow 010\underline{S}1S \Rightarrow 0101\underline{S} \Rightarrow 0101$$

The variable that gets replaced at each step of the derivation is underlined.

Let G be a grammar and $w \in L(G)$.

- A derivation of w with respect to G is a *leftmost derivation* if at every step the leftmost variable gets substituted.
In example 2 above, derivations 1 and 3 are leftmost derivations, whereas 2 is not.
- We say that w is *ambiguous* with respect to G if w has at least 2 leftmost derivations with respect to G . Otherwise we say that w is *unambiguous*.
- A CFG G is said to be *ambiguous* if it generates some string ambiguously. The grammar G_1 above is unambiguous and G_2 is ambiguous.
- A CFL is said to be *inherently ambiguous* if all CFGs that accept the language are ambiguous.

Theorem 15. *Let G be a grammar and $w \in L(G)$. w is unambiguous with respect to G if and only if w has a unique parse tree with respect to G .*

Exercise 17. Prove Theorem 15.

Exercise 18. Give an example of an unambiguous grammar that has at least 2 derivations for some string.

Chapter 9

Chomsky Normal Form

Normal forms are CFGs whose substitution rules have a special form. Usually normal forms are general enough in the sense that any CFL will have a CFG in that normal form. Normal forms have a nice combinatorial structure which are useful in proving properties about CFLs.

9.1 Chomsky Normal Form

A CFG is said to be in *Chomsky Normal Form* (in short, CNF) if the following are true.

1. Every rule is of the form

- $A \longrightarrow BC$, or
- $A \longrightarrow a$,

where A, B, C are variables and a is a terminal.

2. The start variable is not present in the right hand side of any rule.
3. The rule $S \longrightarrow \epsilon$ may be present (depending on whether the language has ϵ or not).

9.1.1 Converting a CFG to a grammar in Chomsky Normal Form

Let $G = (V, \Sigma, P, S)$ be a CFG. Below we give an algorithm to convert G into a CFG in Chomsky Normal Form.

1. **Removing S from RHS of rules**

If S appears on the RHS of some rule, add a new start variable S_0 and the rule $S_0 \longrightarrow S$.

2. **Removing ϵ -rules**

Pick an ϵ -rule $A \longrightarrow \epsilon$ and remove it (where A is not the start variable).

- Now for every occurrence of A on the RHS of some of all rules, add a rule deleting that occurrence of A . If A occurs multiple times on the RHS of a rule, then multiple rules might be added.
- If we have the rule $R \longrightarrow A$, then add $R \longrightarrow \epsilon$, unless the rule $R \longrightarrow \epsilon$ has been removed previously.
- Repeat until no more ϵ -rules remain, except possibly involving the start variable.

Example: Suppose a grammar had the following rules:

$$\begin{aligned} A &\longrightarrow \epsilon \\ B &\longrightarrow uAv \\ C &\longrightarrow u_1Av_1Aw_1 \end{aligned}$$

Then the grammar formed by removing the rule $A \longrightarrow \epsilon$ will have the corresponding set of rules

$$\begin{aligned} B &\longrightarrow uAv \\ C &\longrightarrow u_1Av_1Aw_1 \\ B &\longrightarrow uv && \text{(rule added)} \\ C &\longrightarrow u_1v_1Aw_1 && \text{(rule added with first occurrence of } A \text{ removed)} \\ C &\longrightarrow u_1Av_1w_1 && \text{(rule added with second occurrence of } A \text{ removed)} \\ C &\longrightarrow u_1v_1w_1 && \text{(rule added with both occurrence of } A \text{ removed)} \end{aligned}$$

3. Removing unit rules

Remove a rule $A \longrightarrow B$ and for all rules of the form $B \longrightarrow u$ add the rule $A \longrightarrow u$, unless $A \longrightarrow u$ is a unit rule that has already been removed. Repeat until no more unit rules remain.

Example: Suppose a grammar had the following rules:

$$\begin{aligned} A &\longrightarrow B \\ B &\longrightarrow u \end{aligned}$$

Then the grammar formed by removing the rule $A \longrightarrow B$ will have the corresponding set of rules

$$\begin{aligned} B &\longrightarrow u \\ A &\longrightarrow u && \text{(rule added)} \end{aligned}$$

4. Shortening the RHS

For every rule of the form $A \longrightarrow u_1u_2 \dots u_k$ for $k \geq 3$, where $u_i \in V \cup T$, replace the rule with

$$\begin{aligned} A &\longrightarrow u_1A_1 \\ A_1 &\longrightarrow u_2A_2 \\ A_2 &\longrightarrow u_3A_3 \\ &\vdots \\ &\vdots \\ &\vdots \\ A_{k-2} &\longrightarrow u_{k-1}u_k \end{aligned}$$

Here A_i 's are the new variables added to the grammar.

5. Replacing certain terminals

If there is a rule of the form $A \longrightarrow uv$ where at least one of either u or v is a terminal symbol (say u), then replace the rule $A \longrightarrow uv$ with

$$\begin{aligned} A &\longrightarrow Uv \\ U &\longrightarrow u \end{aligned}$$

where U is a new variable added to the grammar. Repeat until no such rules remain.

9.1.2 An Example – CFG to CNF

Consider the following CFG where S is the start variable:

$$\begin{aligned} S &\longrightarrow ASB \\ A &\longrightarrow aASA \mid a \mid \epsilon \\ B &\longrightarrow SbS \mid A \mid bb \end{aligned}$$

We will convert the above grammar into a grammar in CNF. The rules/variables that get added at each step are shown in **bold** font.

1. Adding a new start variable S_0 , since S appears on RHS of some rules.

$$\begin{aligned} \mathbf{S_0} &\longrightarrow S \\ S &\longrightarrow ASB \\ A &\longrightarrow aASA \mid a \mid \epsilon \\ B &\longrightarrow SbS \mid A \mid bb \end{aligned}$$

2. Eliminating $A \longrightarrow \epsilon$.

$$\begin{aligned} S_0 &\longrightarrow S \\ S &\longrightarrow ASB \mid \mathbf{SB} \\ A &\longrightarrow aASA \mid a \mid \mathbf{aSA} \mid \mathbf{aAS} \mid \mathbf{aS} \\ B &\longrightarrow SbS \mid A \mid bb \mid \epsilon \end{aligned}$$

3. Eliminating $B \longrightarrow \epsilon$.

$$\begin{aligned} S_0 &\longrightarrow S \\ S &\longrightarrow ASB \mid SB \mid \mathbf{AS} \mid \mathbf{S} \\ A &\longrightarrow aASA \mid a \mid aSA \mid aAS \mid aS \\ B &\longrightarrow SbS \mid A \mid bb \end{aligned}$$

4. Eliminating the unit rule $S \longrightarrow S$.

$$\begin{aligned} S_0 &\longrightarrow S \\ S &\longrightarrow ASB \mid SB \mid AS \\ A &\longrightarrow aASA \mid a \mid aSA \mid aAS \mid aS \\ B &\longrightarrow SbS \mid A \mid bb \end{aligned}$$

5. Eliminating the unit rule $B \longrightarrow A$.

$$\begin{aligned} S_0 &\longrightarrow S \\ S &\longrightarrow ASB \mid SB \mid AS \\ A &\longrightarrow aASA \mid a \mid aSA \mid aAS \mid aS \\ B &\longrightarrow SbS \mid bb \mid \mathbf{aASA} \mid \mathbf{a} \mid \mathbf{aSA} \mid \mathbf{aAS} \mid \mathbf{aS} \end{aligned}$$

6. Eliminating the unit rule $S_0 \rightarrow S$.

$$\begin{aligned}
S_0 &\rightarrow \mathbf{ASB} \mid \mathbf{SB} \mid \mathbf{AS} \\
S &\rightarrow ASB \mid SB \mid AS \\
A &\rightarrow aASA \mid a \mid aSA \mid aAS \mid aS \\
B &\rightarrow SbS \mid bb \mid aASA \mid a \mid aSA \mid aAS \mid aS
\end{aligned}$$

7. Adding variable U_1 and rule $U_1 \rightarrow AS$.

$$\begin{aligned}
S_0 &\rightarrow \mathbf{U_1B} \mid SB \mid AS \\
S &\rightarrow \mathbf{U_1B} \mid SB \mid AS \\
A &\rightarrow a\mathbf{U_1A} \mid a \mid aSA \mid \mathbf{aU_1} \mid aS \\
B &\rightarrow SbS \mid bb \mid \mathbf{aU_1A} \mid a \mid aSA \mid \mathbf{aU_1} \mid aS \\
\mathbf{U_1} &\rightarrow \mathbf{AS}
\end{aligned}$$

8. Adding variable U_2 and rule $U_2 \rightarrow aU_1$.

$$\begin{aligned}
S_0 &\rightarrow U_1B \mid SB \mid AS \\
S &\rightarrow U_1B \mid SB \mid AS \\
A &\rightarrow \mathbf{U_2A} \mid a \mid aSA \mid aU_1 \mid aS \\
B &\rightarrow SbS \mid bb \mid \mathbf{U_2A} \mid a \mid aSA \mid aU_1 \mid aS \\
U_1 &\rightarrow AS \\
\mathbf{U_2} &\rightarrow \mathbf{aU_1}
\end{aligned}$$

9. Adding variable U_3 and rule $U_3 \rightarrow aS$.

$$\begin{aligned}
S_0 &\rightarrow U_1B \mid SB \mid AS \\
S &\rightarrow U_1B \mid SB \mid AS \\
A &\rightarrow U_2A \mid a \mid \mathbf{U_3A} \mid aU_1 \mid aS \\
B &\rightarrow SbS \mid bb \mid U_2A \mid a \mid \mathbf{U_3A} \mid aU_1 \mid aS \\
U_1 &\rightarrow AS \\
U_2 &\rightarrow aU_1 \\
\mathbf{U_3} &\rightarrow \mathbf{aS}
\end{aligned}$$

10. Adding variable U_4 and rule $U_4 \rightarrow Sb$.

$$\begin{aligned}
S_0 &\rightarrow U_1B \mid SB \mid AS \\
S &\rightarrow U_1B \mid SB \mid AS \\
A &\rightarrow U_2A \mid a \mid U_3A \mid aU_1 \mid aS \\
B &\rightarrow \mathbf{U_4S} \mid bb \mid U_2A \mid a \mid U_3A \mid aU_1 \mid aS \\
U_1 &\rightarrow AS \\
U_2 &\rightarrow aU_1 \\
U_3 &\rightarrow aS \\
\mathbf{U_4} &\rightarrow \mathbf{Sb}
\end{aligned}$$

11. Adding variables V_1, V_2 and rules $V_1 \rightarrow a, V_2 \rightarrow b$.

$$\begin{aligned}
S_0 &\rightarrow U_1B \mid SB \mid AS \\
S &\rightarrow U_1B \mid SB \mid AS \\
A &\rightarrow U_2A \mid a \mid U_3A \mid \mathbf{V_1U_1} \mid \mathbf{V_1S} \\
B &\rightarrow U_4S \mid \mathbf{V_2V_2} \mid U_2A \mid a \mid U_3A \mid \mathbf{V_1U_1} \mid \mathbf{V_1S} \\
U_1 &\rightarrow AS \\
U_2 &\rightarrow \mathbf{V_1U_1} \\
U_3 &\rightarrow \mathbf{V_1S} \\
U_4 &\rightarrow \mathbf{SV_2} \\
\mathbf{V_1} &\rightarrow \mathbf{a} \\
\mathbf{V_2} &\rightarrow \mathbf{b}
\end{aligned}$$

Chomsky Normal Form of the given grammar is

$$\begin{aligned}
S_0 &\rightarrow U_1B \mid SB \mid AS \\
S &\rightarrow U_1B \mid SB \mid AS \\
A &\rightarrow U_2A \mid U_3A \mid V_1U_1 \mid V_1S \mid a \\
B &\rightarrow U_4S \mid V_2V_2 \mid U_2A \mid U_3A \mid V_1U_1 \mid V_1S \mid a \\
U_1 &\rightarrow AS \\
U_2 &\rightarrow V_1U_1 \\
U_3 &\rightarrow V_1S \\
U_4 &\rightarrow SV_2 \\
V_1 &\rightarrow a \\
V_2 &\rightarrow b
\end{aligned}$$

Exercise 19. Problem 2.14 from textbook.